TNMCORE-OS: Hệ Điều Hành Agile Tích Hợp AI (Version 1.0)

TNMCORE-OS là hệ điều hành phát triển phần mềm tiên tiến nhất, được thiết kế để tối ưu hóa sự cộng tác giữa Human và Agentic AI. Hệ thống vận hành dựa trên triết lý **Spec-Driven Development (SDD)**, biến AI từ công cụ hỗ trợ thành một cộng sự thực thụ trong mọi giai đoạn của SDLC.

Mục lục (Table of Contents)

1. Tầm Nhìn & Triết Lý (Vision & Philosophy)
2. Nguyên Tắc & Kỷ Luật Làm Việc (Core Principles & Disciplines)
3. Giao Thức Vận Hành Gốc (Core Execution Protocol)
4. Cài Đặt & Thiết Lập (Installation & Setup)
5. Hệ Thống Vai Trò AI (AI Roles & Skills)
6. Quy Trình Non-Coding (Intel Construction)
7. Quy Trình Coding (Spec-Driven Development)
8. Cấu Trúc Thư Mục (Directory Structure)
9. Hệ Thống Tri Thức & Quy Định Tài Liệu
10. Hỏi Đáp Thường Gặp (Q&A)
11. Tài Liệu Tham Khảo (References)

---

1. Tầm Nhìn & Triết Lý (Vision & Philosophy)

TNMCORE-OS không chỉ là một Project Codebase, mà là một **"Hệ điều hành cho Team"** (Operating System for Teams). Mục tiêu cốt lõi là chuyển đổi quy trình Agile truyền thống

sang mô hình **AI-Native Agile**:

- **Bộ não thứ 2 (The 2nd Brain):** Hệ lưu trữ tập trung tri thức, quy trình và thuật toán (SSOT). Giảm tải ghi nhớ cho Human, tập trung nguồn lực cho các quyết định chiến lược.
- **AI-Native Agile:** Tích hợp AI vào mọi nhánh của SDLC, từ Discovery Requirements đến Deployment & Ops.

---

2. Nguyên Tắc & Kỷ Luật Làm Việc (Core Principles & Disciplines)

TNMCORE-OS vận hành dựa trên sự kết hợp giữa các nguyên tắc kiến trúc và kỷ luật thực thi nghiêm ngặt:

2.1 Các Nguyên Tắc "Bất Di Bất Dịch"

- **Single Source of Truth (SSOT):** Mọi tri thức nằm tại `docs/`. Tuyệt đối không có ngoại lệ.
- **Dewey Decimal Standard:** Tổ chức tài liệu theo hệ thống phân loại số (Taxonomy).
- **Spec Before Code:** Không Implementation nếu chưa có Specs được chứng thực.
- **Context-Aware Action:** Buộc Agent thực hiện **Discovery** trước khi đưa ra hành động.
- **Identity First (Role Priority):** Xác định **Role** (Mindset) trước khi kích hoạt **Skills** (Tools).

2.2 Kỷ Luật Giao Tiếp & Thực Thi (Agentic Rules)

- **Định danh TNMCORE-OS:** AI luôn tự xưng là **"TNMCORE-OS"** - Trợ lý vận hành toàn năng.
- **Giao thức Plan & Approve:** Bắt buộc trình bày **Step-by-Step Plan** và giải thích logic "Tại sao" trước mọi thay đổi hệ thống.
- **Hybrid Communication:** Ngôn ngữ Tiếng Việt chủ đạo kết hợp thuật ngữ IT (English) để đạt độ chính xác kỹ thuật tối đa.
- **Research-First:** Ưu tiên `search_web` tìm Best Practices quốc tế cho các bài toán quan trọng. Lưu kết quả tại `docs/050-Research/`.

2.3 Tiêu Chuẩn Clean Code & Bảo Mật

- **Tiêu chuẩn Code:** Tuân thủ SOLID, DRY, KISS. Tuyệt đối không sử dụng comment inline.
- **Header Định danh:** Mọi file mới hoặc chỉnh sửa lớn phải có header `// AI Coding` hoặc `# AI Coding`.
- **Bảo mật:** Tuyệt đối không hardcode API Keys/Secrets; chỉ sử dụng biến môi trường (`.env`).
- **Phạm vi (Boundaries):** AI chỉ hoạt động trong thư mục gốc của dự án. Mọi script auto phải nằm trong `/scripts`.

---

3. Giao Thức Vận Hành Gốc (Core Execution Protocol)

Đây là "Hiến pháp tư duy" của TNMCORE-OS, áp dụng cho cả Coding và Non-Coding tasks. AI luôn đảm bảo thực hiện đúng 6 bước:

1. **Discovery (Context Loading):** Nạp bối cảnh từ `docs/` và `knowledge-base/`.
2. **Solution Design:** Phác thảo Architecture và phản biện giải pháp.
3. **Planning:** Lập **Step-by-Step Plan** và chờ User Approval.
4. **Implementation:** Triển khai mã nguồn hoặc nội dung theo Specs.
5. **Verification:** Chạy Tests và đối soát kết quả với Requirement.
6. **Finalize & Learning Loop:** Đóng gói Artifacts và đúc kết kinh nghiệm vào Memory.

> 📖 ***Xem chi tiết:*** *Hướng dẫn thực thi từng bước tại OS-Handbook.md.*

---

4. Cài Đặt & Thiết Lập (Installation & Setup)

4.1 Yêu Cầu Môi Trường

- **IDE Khuyến Nghị: Antigravity**.
- **Node.js:** Phiên bản **20.19.0** trở lên.
- **Package Manager:** `npm`, `pnpm`, hoặc `bun`.

4.2 Cài Đặt Công Cụ

Nếu bạn là DEV hoặc tham gia vào quá trình phát triển CODING, hãy cài đặt **OpenSpec CLI** toàn cục:

```
npm install -g @fission-ai/openspec@latest
```

4.3 Chuẩn Bị Dự Án (Setup)

1. **Clone Project:**

   ```
   git clone git@github.com:tenomad-company/TNM-TEAM-OS.git
   cd team-os
   ```

2. **Cài đặt dependencies:**

   ```
   npm install
   ```

3. **Cấu hình môi trường:** Hệ thống yêu cầu các biến môi trường để tích hợp với Microsoft Teams và ClickUp.

```
cp .env.example .env
# Cập nhật API Keys, Workspace IDs cần thiết
```

---

5. Hệ Thống Vai Trò AI (AI Roles & Skills)

5.1 Nguyên Tắc Vận Hành: "Role-Playing First"

Trong TNMCORE-OS, kỹ năng (Skills) chỉ là công cụ. Để sử dụng công cụ hiệu quả, trước tiên AI phải xác định được **"Mình là ai?"** và **"Mình tư duy như thế nào?"**.

> ⚠️ ***Quy tắc Bắt buộc:*** *Trước khi thực hiện bất kỳ workflow phức tạp nào, User/AI hãy kích hoạt Role tương ứng bằng cách đọc file định nghĩa Role (`.agent/roles/*.md`).*

5.2 Danh Sách Vai Trò (Layered Personas)

TNMCORE-OS phân cấp các vai trò chuyên gia. Mỗi Role có Mindset và bộ Skill riêng biệt.

> 🛠️ ***Sử dụng:*** *Gõ lệnh mẫu:* ***"Hãy đóng vai trò [Tên Role]"*** *trước khi giao Task.*

Role	Định Nghĩa (Persona & Mindset)	Nhiệm Vụ Cốt Lõi
PM	`.agent/roles/product-manager.md`	Quản trị Roadmap, Sprint & ROI.

BA	`.agent/roles/business-analyst.md`	Phân tích nghiệp vụ & Requirements.
Architect	`.agent/roles/architect.md`	Thiết kế hệ thống & Architecture.
Senior AI Engineer	`.agent/roles/senior-ai-engineer.md`	Kiến tạo & tối ưu hóa "Bộ não" AI (Agentic OS).
Engineer	`.agent/roles/software-engineer.md`	Xây dựng mã nguồn (Clean Code).
QA	`.agent/roles/quality-assurance.md`	Gác cổng chất lượng & Testing.

> 📗 ***Học cách làm chủ Roles & Skills:*** *Xem tại OS-Handbook.md.*

6. Quy Trình Non-Coding (Intel Construction)

Quy trình dành cho Non-Software Engineers và các vai trò khác tham gia vào việc xây dựng tài liệu `docs/` hoặc tri thức `knowledge-base/`, chiến lược và đặc tả như BA, PM, Architect, Designer, QA.

6.1 Triết lý: "Identity First - Documentation as Code"

Trong TNMCORE-OS, tri thức không nằm trong đầu con người mà nằm trong hệ thống tài liệu được cấu trúc hóa. Công việc của các vai trò Non-SSD là xây dựng **Nguồn sự thật (SSOT)** để AI Code có thể thực thi chính xác.

6.2 Chu trình vận hành 6 bước (Intel Construction):

Quy trình này áp dụng **Giao Thức Vận Hành Gốc (Phần 3)** để biến các tác vụ phi kỹ thuật thành quá trình "Kỹ nghệ hóa tri thức" (Knowledge Engineering) đa vai trò:

1. **Thu thập & Phân tích (Discovery):**

6. Quy Trình Non-Coding (Intel Construction)

Quy trình dành cho việc xây dựng tài liệu, chiến lược và đặc tả (BA, PM, Architect, Designer, QA).

TNMCORE-OS áp dụng triết lý **"Documentation as Code"**: Tri thức được cấu trúc hóa, có liên kết chặt chẽ và định danh số (Dewey).

> 📘 ***Hướng dẫn SOP:*** *Xem chi tiết luồng Intel Construction tại OS-Handbook.md.*

7. Quy Trình Coding (Spec-Driven Development)

Quy trình dành cho Software Engineers vận hành theo kỷ luật **Spec-Driven Development (SDD)** thông qua OpenSpec (OPSX).

> 🛠️ ***Thực thi:*** *Sử dụng các lệnh* `/opsx-...` *để bắt đầu chu kỳ:* ***Spec-First, Code-Later****.*

> 💻 ***Hướng dẫn SOP:*** *Xem chi tiết luồng Coding thực chiến tại OS-Handbook.md.*

---

8. Cấu Trúc Thư Mục (Directory Structure)

Dự án được tổ chức theo mô hình hiện đại, tối ưu cho cả AI và Con người:

```
team-os/
├── .agent/                  # Bộ não AI: Gồm Skills, Workflows và Rules
├── docs/                    # Tài liệu dự án cụ thể (Project Docs) - Dewey Dec
│   ├── 000-Index.md         # Mục lục tài liệu tổng thể
│   ├── 010-Planning/        # Kế hoạch & Sprints
│   ├── 020-Requirements/    # PRD, BRD, Use Cases
│   ├── 022-User-Stories/    # Backlog, Epics, Sprint hiện tại
│   ├── 030-Specs/           # Đặc tả kỹ thuật (API, Architecture, Schema)
│   ├── 035-QA/              # Kiểm thử (Test Plans, Test Cases, Reports)
│   ├── 040-Design/          # Thiết kế (Design System, Assets, Specs UI)
│   ├── 050-Research/        # Nghiên cứu người dùng
│   └── ...
├── knowledge-base/           # Hệ thống tri thức nền tảng (Base Knowledge)
│   ├── 00-Index.md          # Mục lục hệ thống tri thức
│   ├── 01-Metas/            # Từ điển thuật ngữ (Glossary/Ontology)
│   ├── 10-Technical/        # Tiêu chuẩn Coding, Kiến trúc Agent
│   ├── 20-Project/          # Quản trị (PMP), Governance, SOP
│   ├── 30-Domain/           # Kiến thức nghiệp vụ đặc thù
│   ├── 40-Memory/           # Học hỏi (Lessons Learned), Feedback Loops
│   └── 99-Templates/        # Mẫu tài liệu, Prompt Standards
├── openspec/                # Quy trình Spec-Driven Development (SDD)
├── src/                     # Mã nguồn chính của dự án (Application Codebase)
├── OS-Handbook.md           # Cẩm nang vận hành chi tiết (Manual)
├── OS-Cheatsheet.md         # Bảng tra cứu nhanh lệnh & quy trình
└── README.md                # Tài liệu hướng dẫn này
```

9. Hệ Thống Tri Thức & Quy Định Tài Liệu

Thư mục `knowledge-base/` là "Bộ nhớ dài hạn" của hệ thống, chứa các tri thức nền tảng không thay đổi theo từng task nhưng áp dụng cho toàn dự án.

9.1 Vai Trò của AI Agent (Kỷ luật Dewey)

Mọi AI Agent khi bắt đầu một nhiệm vụ **BẮT BUỘC** phải:

1. **Tra cứu Tiêu chuẩn:** Kiểm tra `10-Technical/Coding-Standards.md` để đảm bảo code viết ra tuân thủ Style Guide.
2. **Sử dụng Ngôn ngữ chung:** Tham chiếu `01-Metas/Glossary.md` để dùng đúng thuật ngữ chuyên môn.
3. **Áp dụng Prompt Engineering:** Tuân thủ khung **CO-STAR** tại `99-Templates/` để cải thiện chất lượng suy luận.
4. **Học hỏi kinh nghiệm:** Đọc `40-Memory/` để tránh các lỗi đã từng xảy ra trong lịch sử.

9.2 Phân Loại Dewey cho Knowledge Base

- **01 - 09:** Tri thức về siêu dữ liệu và ngôn ngữ.
- **10 - 19:** Tiêu chuẩn kỹ thuật và thực thi code.
- **20 - 29:** Quy trình quản lý và vận hành dự án.
- **30 - 39:** Kiến thức chuyên môn về lĩnh vực (Domain).
- **40 - 49:** Lưu trữ kinh nghiệm và phản hồi.

---

10. Hỏi Đáp Thường Gặp (Q&A)

H: Sự khác biệt giữa `docs/` và `knowledge-base/` là gì?

- **Đ:** `docs/` chứa các tài liệu **đặc thù của dự án** (như PRD của tính năng X, Design của App Y). `knowledge-base/` chứa các **nguyên tắc nền tảng** (như Cách viết code chuẩn, Cách quản lý dự án chuẩn) áp dụng chung cho mọi tính năng.

H: Làm sao để cập nhật kiến thức cho AI?

- **Đ:** Khi gặp một bài học mới, hãy cập nhật vào `40-Memory/After-Action-Review.md`. AI sẽ tự động đọc file này trong các task tương lai để cải thiện hành vi.

---

11. Tài Liệu Tham Khảo (References)

1. OS-Handbook (Cẩm nang vận hành)
2. OS-Cheatsheet (Tra cứu nhanh)
3. Hướng dẫn cài đặt OpenSpec chi tiết
4. Tìm hiểu về OPSX Workflow
5. Các khái niệm cốt lõi của OpenSpec
6. Quy trình Development (Workflows)
7. Mục lục hệ thống Tri thức

Tài liệu được khởi tạo và quản lý bởi TNMCORE-OS Antigravity - Hệ Điều Hành Agile AI-Native.